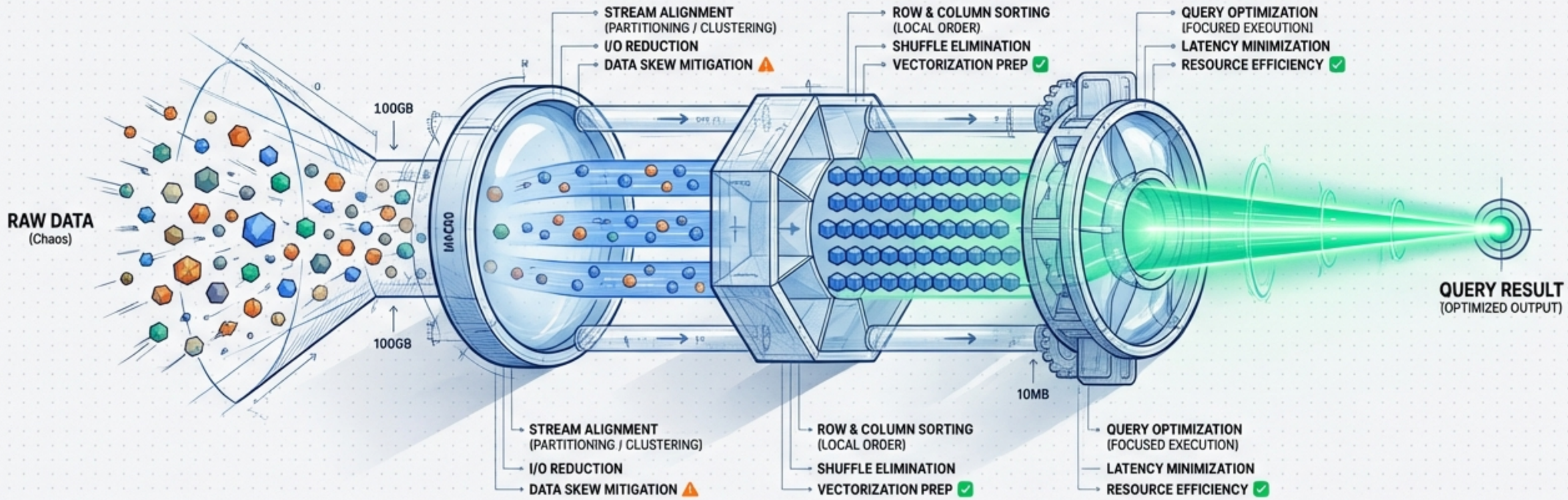
The Optimisation Telescope: From Storage Layout to Query Execution

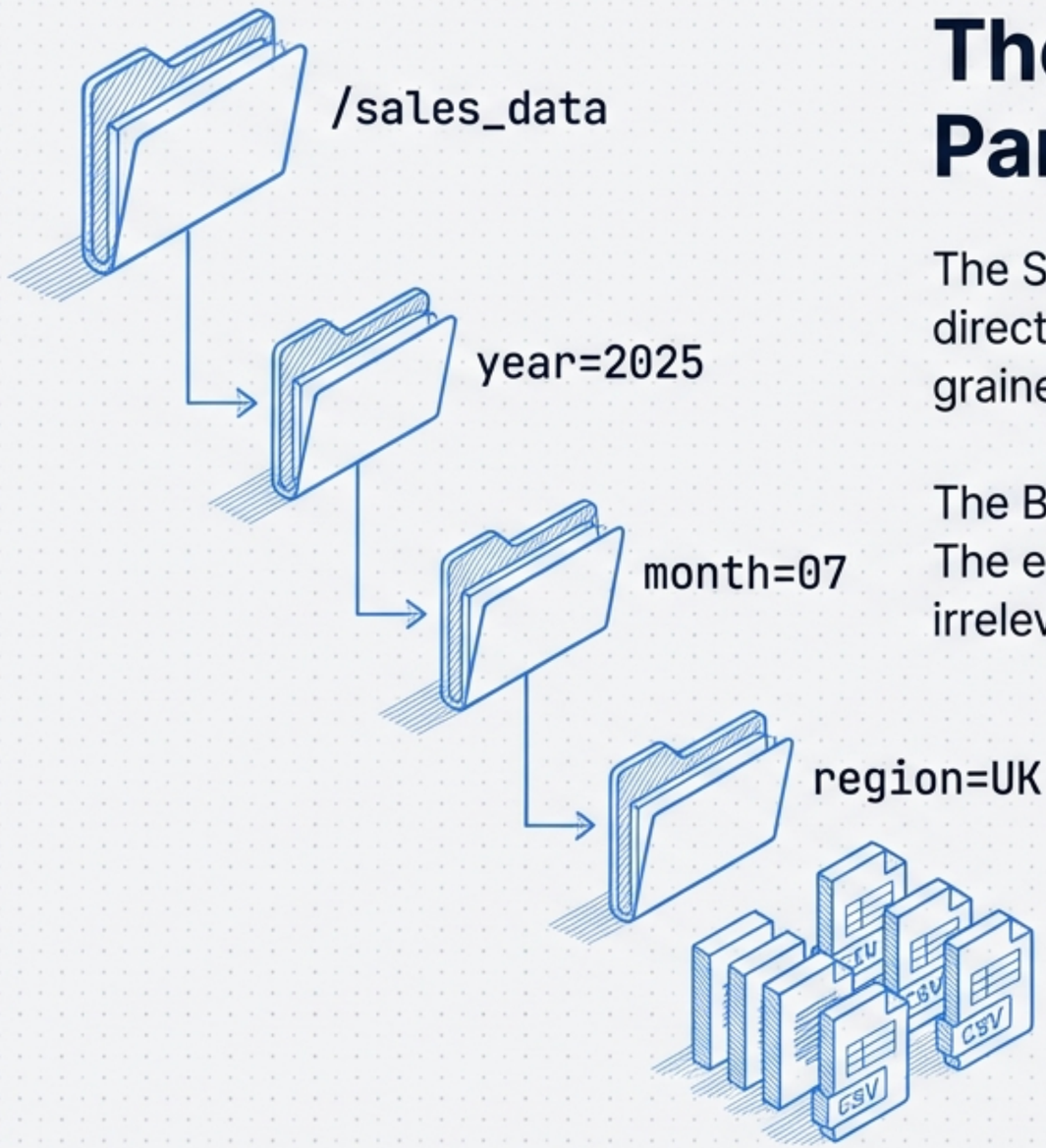
A structural approach to minimising I/O, eliminating shuffles, and solving data skew in modern dataflow systems.



Safety Orange (Hex #F97318): Alerts/Bottlenecks
Blueprint Blue (Hex #3B82F6): Structure/Storage
Signal Green (Hex #108981): Optimization/Success

DATAFLOW OPTIMIZATION PIPELINE

Synthesising insights from Delta Lake, Snowflake, and Sort Merge Bucket research.



The Foundation: Partitioning

The Strategy: Grouping data into directories based on coarse-grained keys.

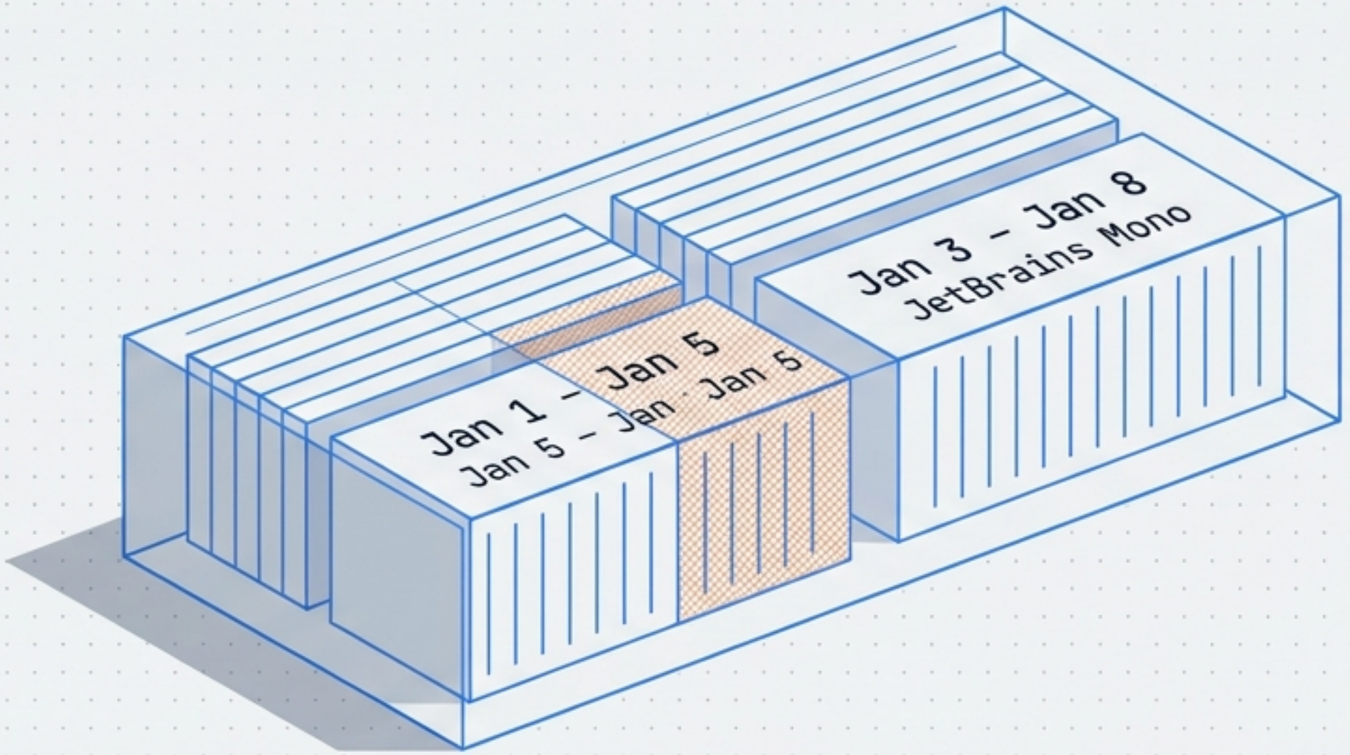
The Benefit: Partition Pruning. The engine skips entire directories irrelevant to the query.

There is no free lunch.

Directory Overhead: High cardinality columns (e.g., user_id) create millions of directories.

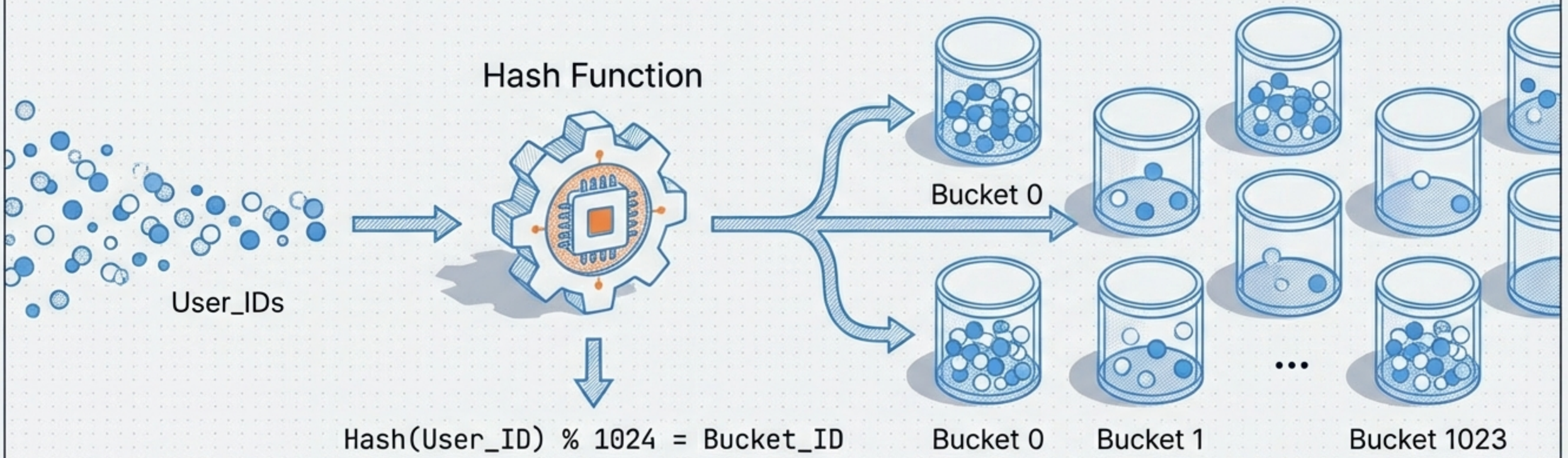
The Small File Problem: Updating partitioned tables can result in fragmented files <1GB, increasing metadata overhead overhead and slowing scanning.

The Macro Evolution: Snowflake Micro-partitions



Size	Structure	Metric
Body 50 MB – 500 MB (uncompressed) Inter and JetBrains Mono	Body Derived automatically; no static definition required. Inter	Body Clustering Depth: Measures the average depth of overlapping partitions. Lower depth = better pruning. Inter

The Micro View: Bucketing (The Hash Strategy)



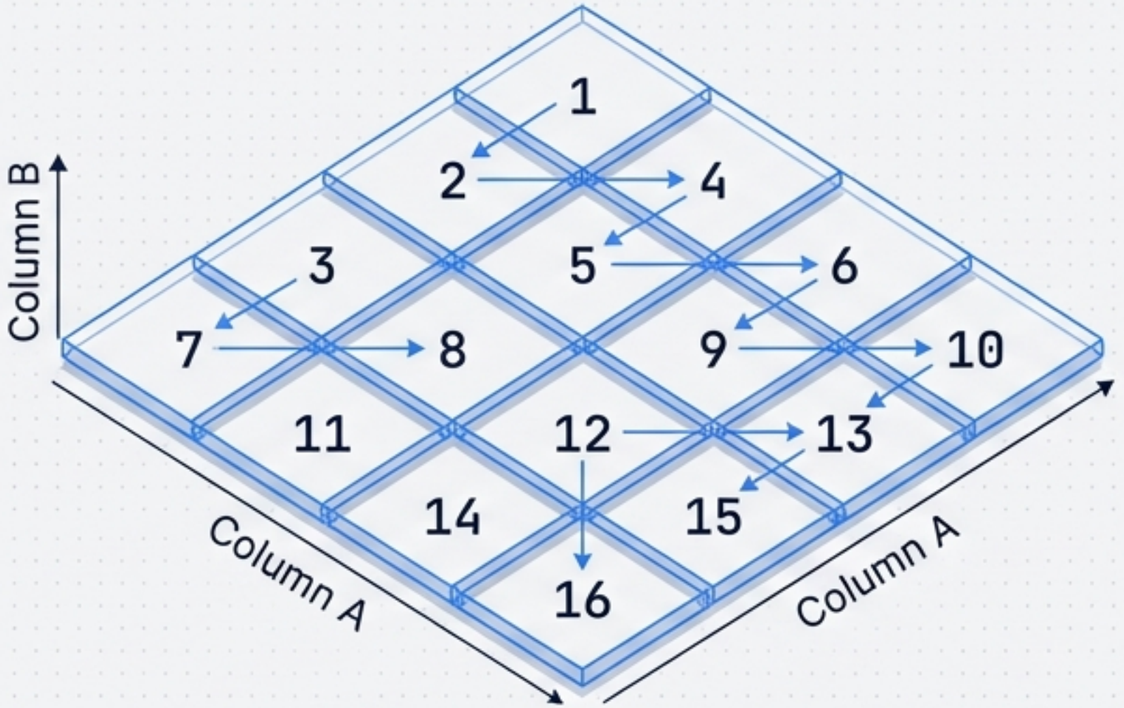
Core Concept

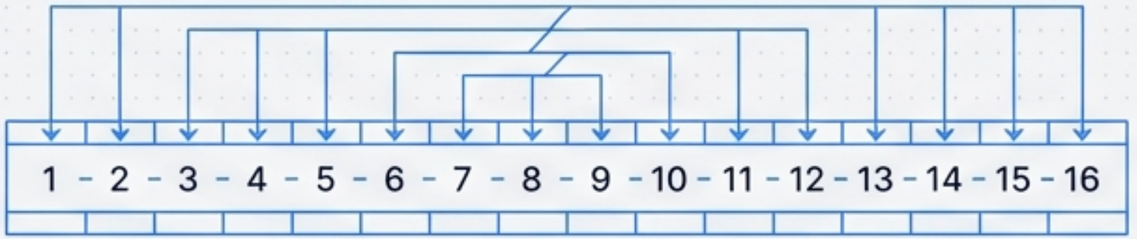
Distributing data into fixed file groups based on a hash of the join key.

Use Cases

- High-cardinality columns where directory partitioning fails.
- Sampling: Ensures all records for a specific user are in the same file.
- Joins: Pre-aligns data for downstream merging (Map-side joins).

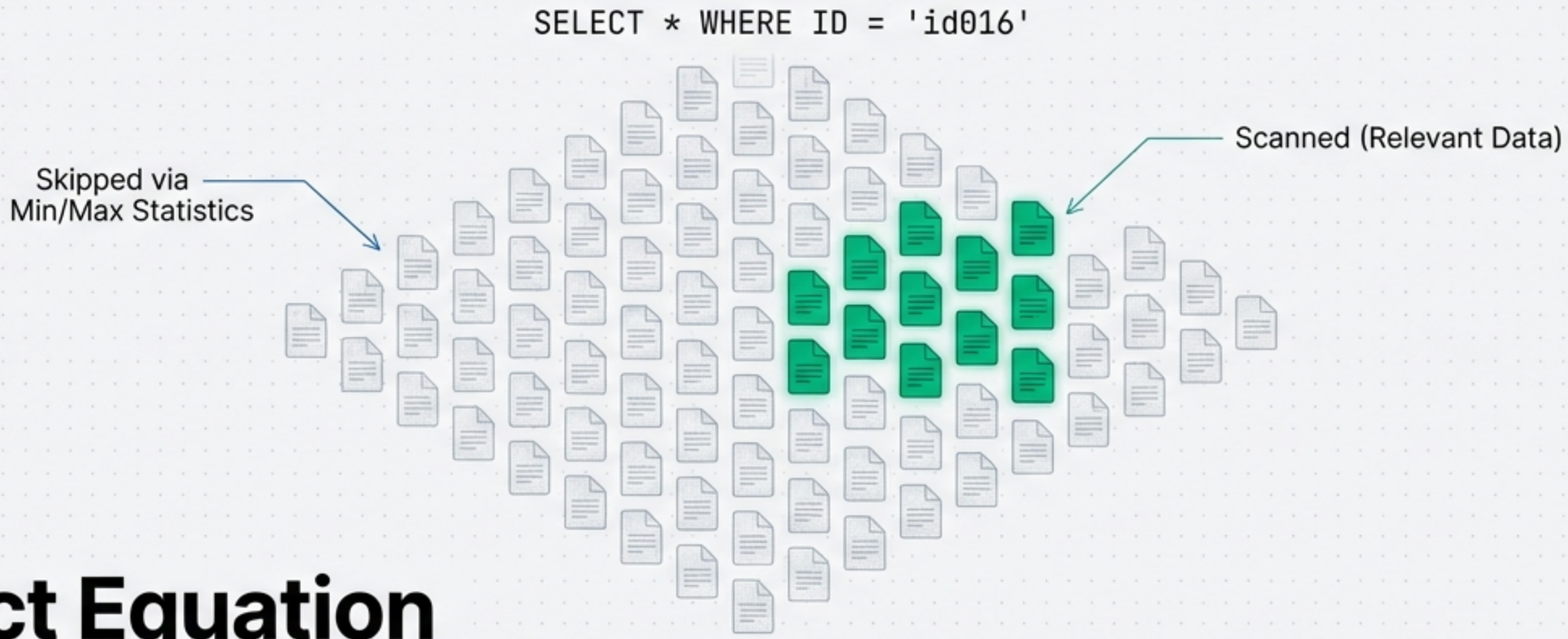
Z-Ordering: Multi-Dimensional Locality





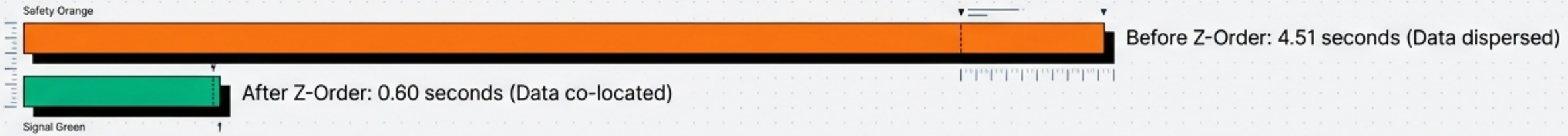
- **The Problem:** Simple sorting only optimizes the first column (Prefix search).
- **The Solution:** Interleaves bits from multiple columns to map multi-dimensional data to linear storage.
- **Benefit:** Co-locates related information. If a query filters by Column A **AND** Column B, the engine **skips massive chunks of data.**

The Pruning Power of Metadata

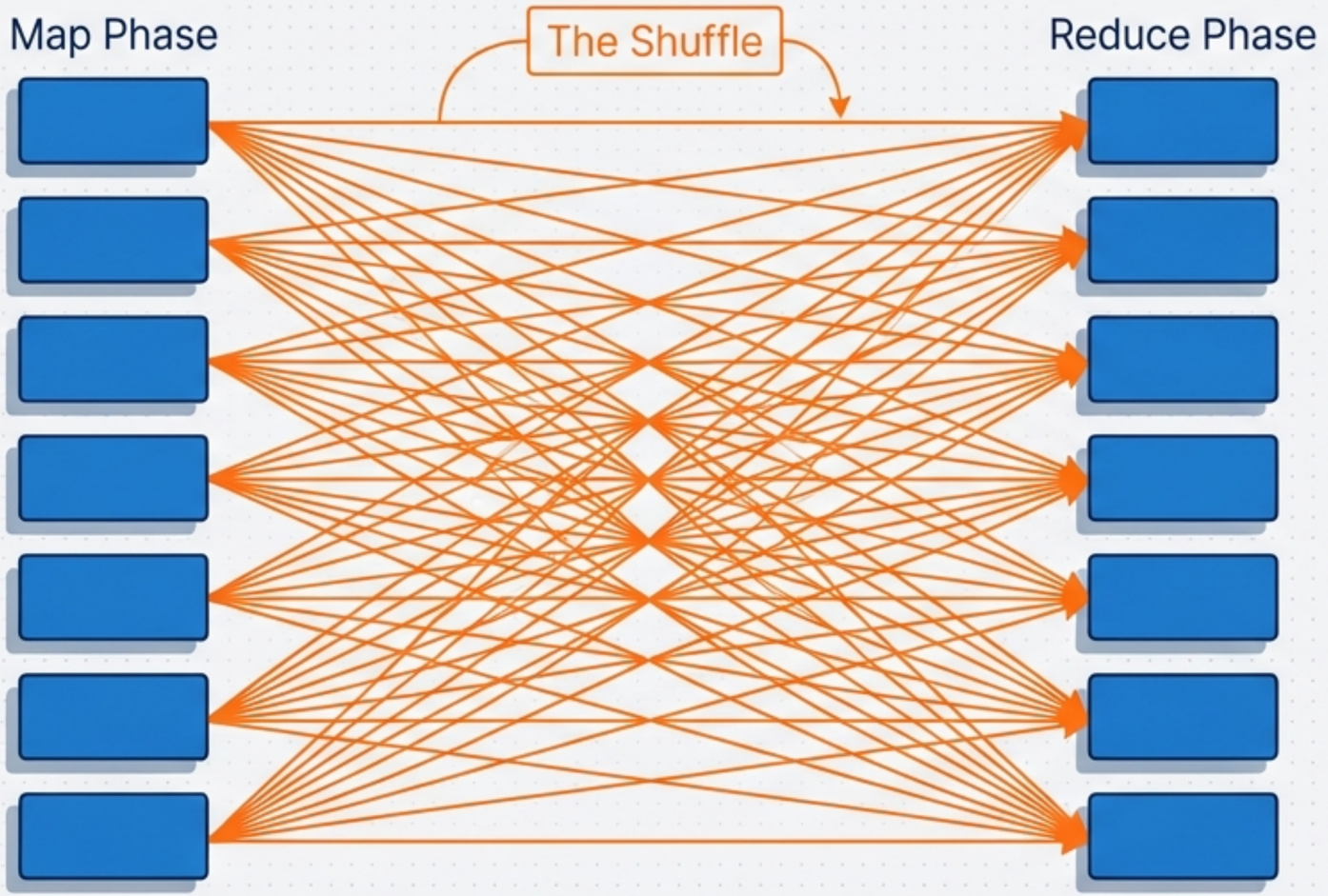


Impact Equation

Min/Max Statistics + Physical Layout (Z-Order) = Massive I/O Reduction



The Bottleneck: Distributed Shuffles



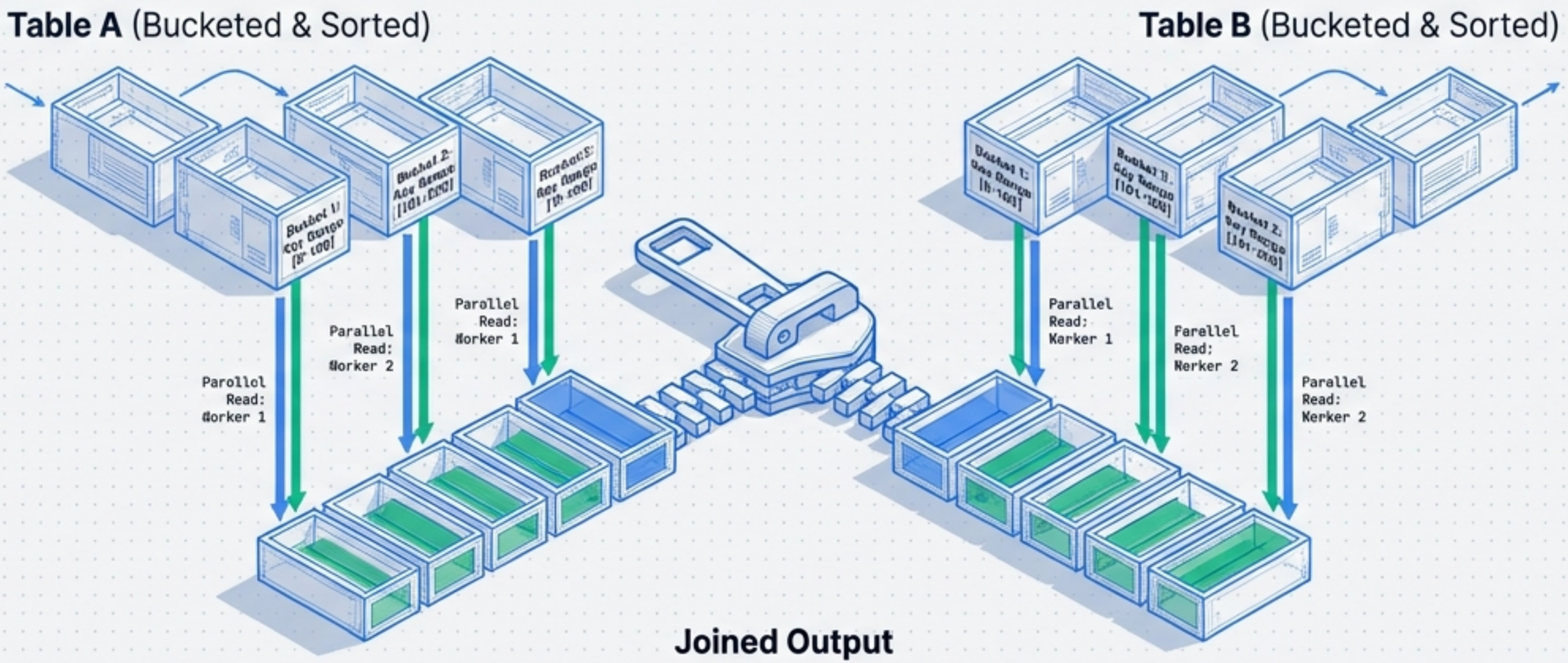
Insight:

Moving data across the network is the most expensive operation in a distributed system.

The Cost:

- **Serialization:** Converting memory objects to bytes.
- **Network I/O:** Orders of magnitude slower than memory.
- **Stragglers:** The entire pipeline waits for the slowest transfer.

The Solution: Sort Merge Buckets (SMB)



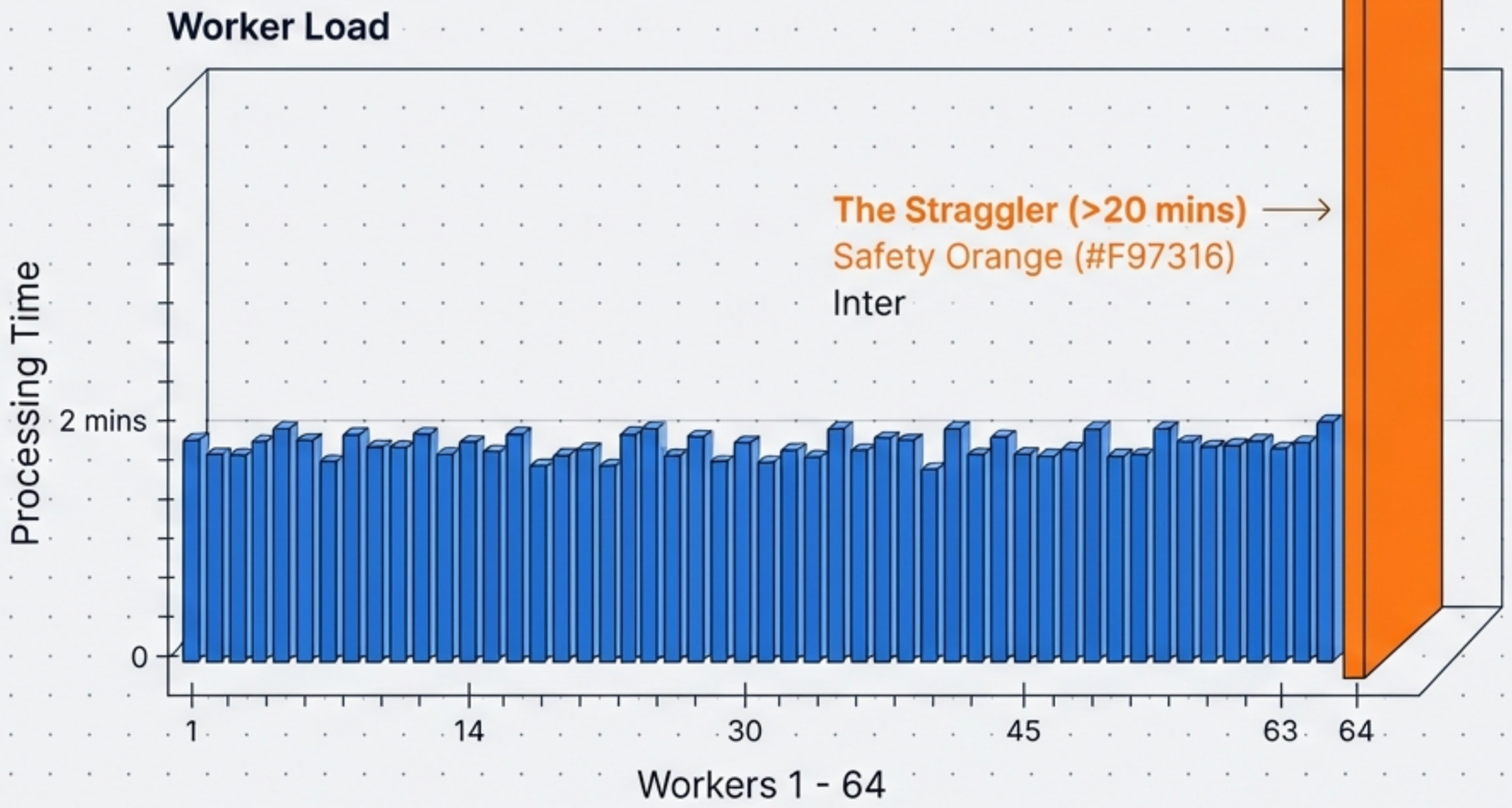
Pre-processing: Writing data into sorted, bucketed files.

Execution: Map-side merge. Worker 1 reads Bucket 1 from Table A and Bucket 1 from Table B. No network exchange required.

Use Case: Repeated Joins (e.g., Decryption Pipelines). The cost of sorting once is amortized over every subsequent read.

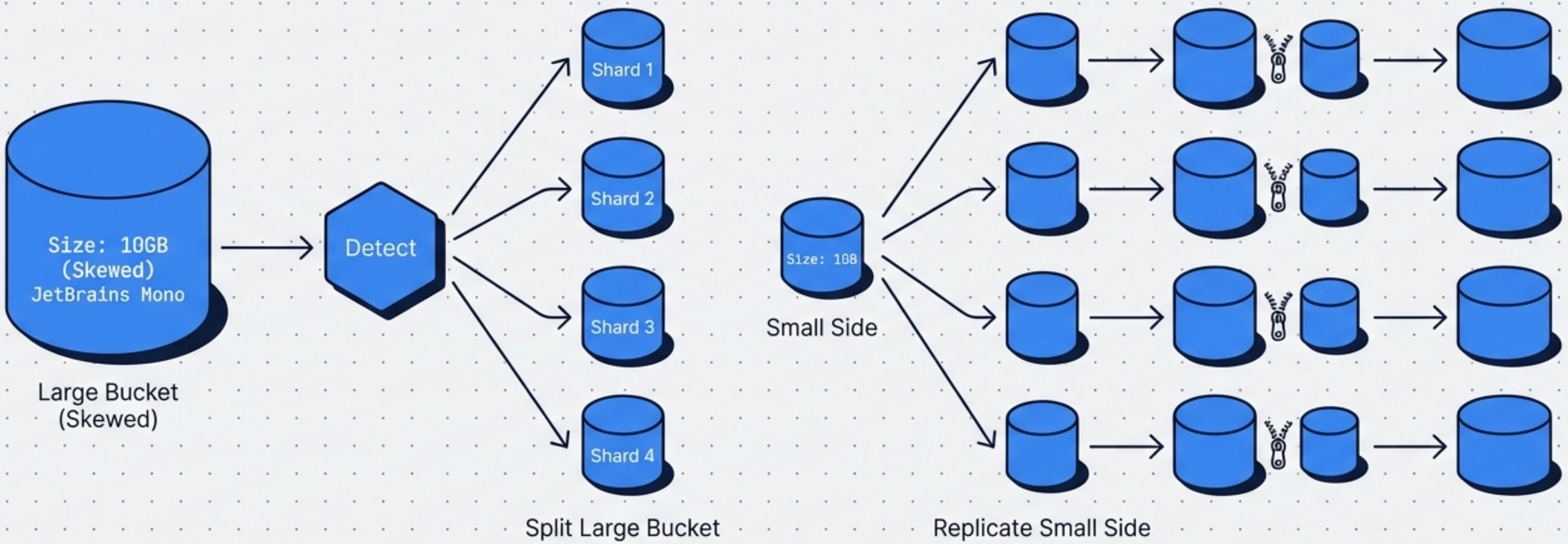
The Villain: Data Skew

The wagon moves as fast as the slowest horse.



- **Data Skew:** Hot keys (e.g., one user has 10M events, others have 10).
- **Processing Skew:** One partition takes disproportionately longer to process.
- **Result:** CPU resources on idle workers are wasted while waiting for the straggler.

The Hero: Skew-Adjusted SMB



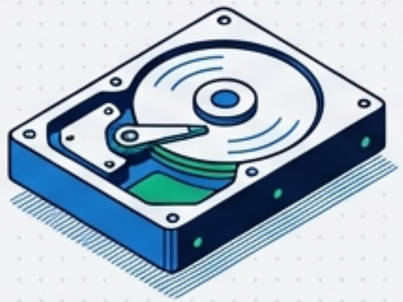
Target Bucket Size: Buckets are sized dynamically.
Sharding & Replication: Large buckets are split; the corresponding small-side data is broadcast only to the relevant workers.
Result: Robustness against extreme skew where standard joins fail.

Helvetica Now Display

Real-World Impact: The Decryption Pipeline

Results Dashboard

Storage

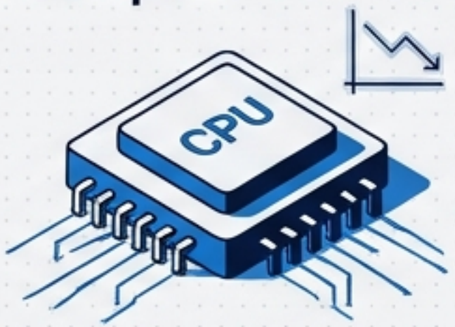


Compression Improved

1.5TB → 1.1TB

Better delta encoding due to sorting.

Compute



CPU Usage

SMB uses fewer CPU resources after just 2 join operations.

Network

Shuffle Volume

Reduced network traffic significantly after 4 operations.

Resilience

Skew Handling

Handles Zipfian distributions where baseline joins crash.

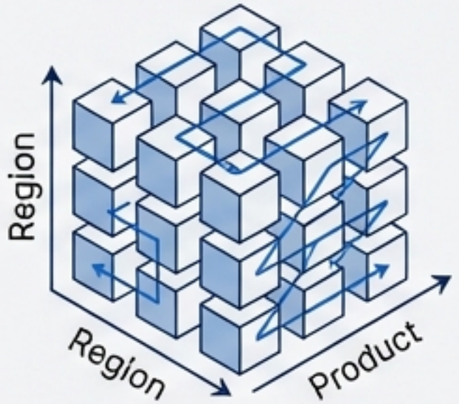
Case Study: Spotify Event Delivery. Billions of events, repeated joins to decrypt user data.

The Trade-Off Matrix: Choosing the Right Tool

Partitioning

Best For: Date/Time filtering.

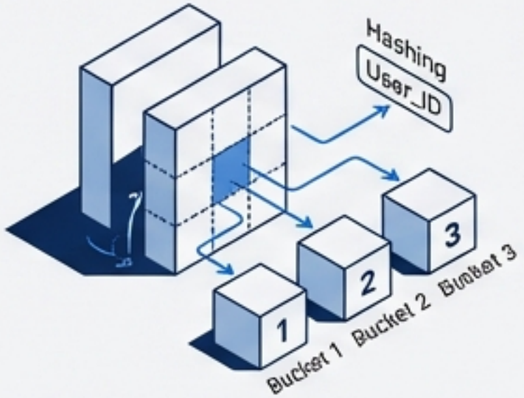
Risk: Small files if cardinality is high.



Z-Ordering

Best For: Multi-column filtering (e.g., Region + Product).

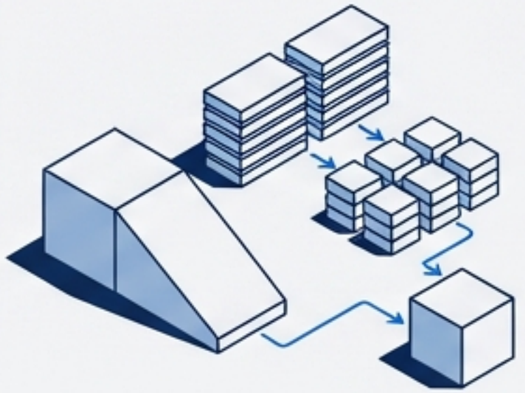
Risk: High compute cost to write/optimize.



Bucketing

Best For: High-cardinality Joins (e.g., User_ID).

Risk: Fixed bucket counts can be inflexible.



SMB

Best For: Repeated Joins on large, skewed datasets.

Risk: Upstream sorting cost.

Summary & Best Practices

Golden Rules

1 Size Matters

Do not partition tables smaller than 1 TB.

JetBrains Mono

Ensure optimal table size before implementation.

2 Avoid Fragmentation

Ensure partitions contain at least 1 GB of data.

JetBrains Mono

Reduce overhead by maintaining minimum partition sizes.

4 Invest Upstream

Pre-sorting and bucketing (SMB) pays pays dividends if data is tata is joined repeatedly.

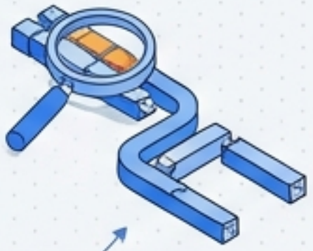
Offset downstream compute costs with strategic data preparation.

3 Match the Pattern

Align partition/clustering keys with your most frequent query predicates.

Drastically improve query performance by matching data layout to access patterns.

JetBrains Mono



5 Beware the Skew

Design for the straggler, not the average worker.

JetBrains Mono

Anticipate and mitigate performance bottlenecks caused by data skew.